

HW6

Wang Xiyu A0282500R

Computability

In the first part of this homework, you will be doing some warm-up exercises relevant to topics in computability.

Turing Machines Syntax. Let us first briefly recall the syntax of the Turing machine. We will work with single tape Turing machines.¹ A Turing Machine is a tuple

$$M = (Q, q_{\text{init}}, q_{\text{acc}}, q_{\text{rej}}, \Gamma, \delta),$$

where

- Q is the (finite) set of states of M .
- $q_{\text{init}} \in Q$ is the initial state of M .
- $q_{\text{acc}} \in Q$ is the unique accepting state of M .
- $q_{\text{rej}} \in Q$ is the unique rejecting state of M ; this is sometimes omitted.
- Γ is the (finite) tape alphabet. We assume that this also includes the blank symbol: $\epsilon \in \Gamma$. We will use

$$\Gamma^\neq = \Gamma \setminus \{\epsilon\}$$

to denote the set of tape symbols other than the blank symbol.

•

$$\delta : Q \times \Gamma \times \Gamma \rightarrow Q \times \{L, R, S\} \times \Gamma \times \{L, R, S\}$$

is the transition function of M .

Given a state $q \in Q$, tape symbol $\sigma_{\text{Inp}} \in \Gamma$ at the input tape head and $\sigma_{\text{work}} \in \Gamma$ at the work tape head, the transition function returns a tuple

$$(q', m_{\text{Inp}}, \sigma'_{\text{work}}, m_{\text{work}}) = \delta(q, \sigma_{\text{Inp}}, \sigma_{\text{work}}).$$

Based on this, the Turing machine changes its state from q to q' , moves the input tape head (resp. work tape head) left/right/nowhere depending upon whether m_{Inp} (resp. m_{work}) is L , R or S , and writes the symbol σ'_{work} at the cell corresponding to the current work tape head.

Accepting inputs. A Turing machine M halts on input $w \in (\Gamma^\neq)^*$ if the run of M on w reaches either the state q_{acc} or the state q_{rej} . An input w is accepted by M if the run (which may be finite or infinite) of M on w reaches the state q_{acc} ; otherwise w is said to be not accepted by M . The language accepted by a Turing Machine M is

$$L(M) = \{w \mid w \text{ is accepted by } M\}.$$

RE and REC languages. A language $L \subseteq (\Gamma^\neq)^*$ is said to be recursively enumerable if there is a Turing machine M such that $L = L(M)$. The set² of recursively enumerable languages is denoted RE. A language L is said to be recursive if there is a Turing machine M such that $L = L(M)$ and further, M halts on every input (whether or not that input is accepted or rejected by M). The class of recursive languages is called REC. The class REC is also the class of decidable languages (or decidable problems). A problem/language not in REC is said to be undecidable. A problem/language is said to be semi-decidable if it is in RE but not in REC.

¹For every k -tape Turing machine M , there is a 1-tape Turing machine M' such that $L(M) = L(M')$.

²Sets of languages are also sometimes referred to as classes of languages, to avoid the confusion arising from the fact that languages themselves are sets (of strings).

Computable Functions. Turing machines can also sometimes have a dedicated output tape, which is write-only. In this case, the transition function also describes what symbol to write on the current cell of the output tape and where to move the output tape head. We will skip formalizing this. Any such Turing Machine M defines a partial function as follows:

$$f_M(w) = \begin{cases} w' & \text{if } M \text{ halts on } w \text{ and writes } w' \text{ on the output tape,} \\ \text{undef} & \text{otherwise (i.e. } M \text{ does not halt on } w). \end{cases}$$

A total function

$$f : (\Gamma^\neq)^* \rightarrow (\Gamma^\neq)^*$$

is said to be a computable function if there is a Turing machine M such that M halts on every input and $f = f_M$.

Many-one Reductions and RE-hardness. A language $A \subseteq (\Gamma^\neq)^*$ many-one reduces to a language $B \subseteq (\Gamma^\neq)^*$, denoted

$$A \leq_m B,$$

if there is a computable function

$$f : (\Gamma^\neq)^* \rightarrow (\Gamma^\neq)^*$$

such that for every $w \in (\Gamma^\neq)^*$, we have

$$w \in A \iff f(w) \in B.$$

In this case, the function f is called a reduction from A to B . A language L is said to be RE-hard if for every language $L' \in \text{RE}$, we have

$$L' \leq_m L.$$

A language L is said to be RE-complete if $L \in \text{RE}$ and L is RE-hard.

Now, consider the following languages:

$$\text{MP} = \{\langle M, w \rangle \mid w \in L(M)\}$$

$$\overline{\text{MP}} = \{\langle M, w \rangle \mid w \notin L(M)\}$$

$$K = \{\langle M \rangle \mid \langle M \rangle \in L(M)\}$$

$$\overline{K} = \{\langle M \rangle \mid \langle M \rangle \notin L(M)\}.$$

In the above, $\langle M, w \rangle$ (resp. $\langle M \rangle$) is an encoding of the pair (M, w) (resp. encoding of the machine M) as a string.

Question 0.1. 1 Establish the following for the above languages:

1. $\text{MP} \in \text{RE}$. (Hint: Define the Universal Turing machine and show that it accepts MP.)
2. $K \in \text{RE}$.
3. $\overline{K}$ is not in RE. (Hint: Use a diagonalization argument.)
4. MP is RE-hard, and thus MP is RE-complete.
5. $\overline{\text{MP}}$ is not in RE via a many-one reduction from $\overline{K}$.

Proof. Consider the following TM construction:

```

M'(M, w):
  b := M(w)
  return b

```

□

Proof. Consider the following construction

```

M'(M):
  b := M(⟨M⟩)
  return b

```

□

Proof. We show that $K \notin REC$ first. We use the same contradiction proof in showing Halting problem is not in REC to show that $K \notin REC$. Suppose $\bar{K} \in RE, K \in RE \wedge \bar{K} \in RE \implies K \in REC$, contradiction. □

Proof.

$$L_{Halt} \leq_m MP$$

Consider the following function:

$$f(\langle M, w \rangle) = \langle M', w \rangle$$

where we define M' as

```

M'(w):
  M(w)
  return true

```

clearly, $\langle M, w \rangle \in L_{Halt} \iff \langle M', w \rangle \in MP$ □

Proof. Consider function

$$f(\langle M \rangle) = \langle M', \varepsilon \rangle$$

Where M' is the same as previous question. □

Church–Turing Theorem

In this section, we will work towards proving the undecidability of the validity problem for FOL. Let us first formally define the language of all valid formulae:

$$\text{VALID}_{\text{FOL}} = \{\varphi \mid \varphi \text{ is a FOL formula and } \emptyset \models \varphi\}.$$

Theorem 0.2 (Church and Turing). $\text{VALID}_{\text{FOL}}$ is *RE-hard* (and thus undecidable).

In order to establish Theorem 1, we show that the language MP many-one reduces to the language $\text{VALID}_{\text{FOL}}$.

Lemma 0.3.

$$MP \leq_m \text{VALID}_{\text{FOL}}.$$

In order to establish the reduction for Lemma 1, we show that there is a computable procedure to translate every string w to a well-formed FOL formula φ_w such that

$$w \in MP \iff \varphi_w \in \text{VALID}_{\text{FOL}}.$$

The formula we construct will depend upon the word w and the Turing Machine M such that $MP = L(M)$; let this Turing Machine be

$$M = (Q, q_{\text{init}}, q_{\text{acc}}, q_{\text{rej}}, \Gamma, \delta).$$

Let us first describe our signature $\Sigma = (C, F, R)$. Our signature will use symbols that correspond to different parts of the Turing Machine.

Constant Symbols.

$$C = \{0\} \cup \{\mathbf{q} \mid q \in Q\} \cup \{\gamma \mid \gamma \in \Gamma\}.$$

- The constant symbol 0 will be used to denote the number 0, which we will use to encode the initial configuration of the Turing machine.
- For every state $q \in Q$, there is a constant symbol $\mathbf{q}$.
- For every tape symbol $\gamma \in \Gamma$, there is a constant symbol γ .

Function Symbols.

$$F = \{s, \text{state@}, \text{inpHead@}, \text{inpCell}, \text{workHead@}, \text{workCell}\}.$$

- The function symbol s is the successor function and $\text{arity}(s) = 1$.
- The function symbol state@ will be used to denote the state of the Turing Machine at any particular time step; $\text{arity}(\text{state@}) = 1$.
- The function symbols inpHead@ and workHead@ will be used to denote the position of the head of the input and work tapes; both have arity 1.
- The function symbols inpCell and workCell will be used to denote the contents of a particular cell of the input and work tapes; both have arity 2.

Relation Symbols. We do not need any relation symbols for our encoding, so

$$R = \emptyset$$

suffices.

The formula φ_w will be a sentence (no free variables) and will have the following form:

$$\varphi_w \equiv (\varphi_{\text{successor}} \wedge \varphi_{\text{consistent}} \wedge \varphi_{\text{init}} \wedge \varphi_{\text{transition}}) \rightarrow \varphi_{\text{accept}}.$$

Let us describe each of the sub-formulae $\varphi_{\text{successor}}$, $\varphi_{\text{consistent}}$, φ_{init} , $\varphi_{\text{transition}}$ and φ_{accept} below.

Successor.

$$\varphi_{\text{successor}} \equiv (\forall x \neg(0 = s(x))) \wedge (\forall x \forall y (s(x) = s(y) \rightarrow x = y)).$$

Lemma 0.4. Let $\mathcal{A} = (U, I)$ be a Σ -structure such that $\mathcal{A} \models \varphi_{\text{successor}}$. Then, there is a subset $U' \subseteq U$ that is isomorphic to $\mathbb{N}$. That is,

$$U' = \{u_0, u_1, \dots\}$$

(where each u_i is distinct), $u_0 = I(0)$, and for each $i \in \mathbb{N}$,

$$u_i = I(s^i(0)),$$

where

$$s^i(0)$$

is shorthand for

$$\underbrace{s(s(\dots s(0) \dots))}_{i \text{ times}}.$$

Question 0.5. 2 Prove Lemma 2.

Consistency. The formula $\varphi_{\text{consistent}}$ talks about some basic consistency properties that we expect our structures to satisfy:

$$\begin{aligned} \varphi_{\text{consistent}} \equiv & \bigwedge_{\substack{q \neq q' \\ q, q' \in Q}} \mathbf{q} \neq \mathbf{q}' \wedge \bigwedge_{\substack{\gamma \neq \gamma' \\ \gamma, \gamma' \in \Gamma}} \gamma \neq \gamma' \\ & \wedge \forall t \left(\bigvee_{q \in Q} \text{state@}(t) = \mathbf{q} \right) \wedge \forall t \forall i \left(\bigvee_{\gamma \in \Gamma} \text{inpCell}(t, i) = \gamma \right) \wedge \forall t \forall i \left(\bigvee_{\gamma \in \Gamma} \text{workCell}(t, i) = \gamma \right). \end{aligned}$$

Initial Configuration.

$$\varphi_{\text{init}} \equiv \text{state@}(0) = \mathbf{q}_{\text{init}} \wedge \text{inpHead@}(0) = 0 \wedge \text{workHead@}(0) = 0$$

$$\wedge \bigwedge_{i=0}^{|w|-1} (\text{inpCell}(0, s^i(0)) = \mathbf{w}[i])$$

$$\wedge \left(\forall x \left(\bigwedge_{i=0}^{|w|-1} \neg(x = s^i(0)) \rightarrow \text{inpCell}(0, x) = \right) \right) \wedge \forall x \text{workCell}(0, x) = .$$

Transition.

$$\varphi_{\text{transition}} \equiv \bigwedge_{\substack{q \in Q \\ \sigma_{\text{Inp}} \in \Gamma \\ \sigma_{\text{work}} \in \Gamma}} \varphi_{(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})}.$$

For fixed $(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})$, suppose

$$(q', m_{\text{Inp}}, \sigma'_{\text{work}}, m_{\text{work}}) = \delta(q, \sigma_{\text{Inp}}, \sigma_{\text{work}}).$$

Then

$$\varphi_{(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})} \equiv \forall t \left(\text{state}@ (t) = \mathbf{q} \wedge \text{inpCell}(t, \text{inpHead}@ (t)) = \sigma_{\text{Inp}} \wedge \text{workCell}(t, \text{workHead}@ (t)) = \sigma_{\text{work}} \right) \rightarrow \psi_{(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})}.$$

The formula $\psi_{(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})}$ is:

$$\begin{aligned} \psi_{(q, \sigma_{\text{Inp}}, \sigma_{\text{work}})} &\equiv \text{state}@ (s(t)) = \mathbf{q}' \wedge \alpha_{m_{\text{Inp}}} \wedge \beta_{m_{\text{work}}} \\ &\wedge \forall x \text{inpCell}(s(t), x) = \text{inpCell}(t, x) \wedge \text{workCell}(s(t), \text{workHead}@ (t)) = \sigma'_{\text{work}} \\ &\wedge \forall x (\neg(x = \text{workHead}@ (t)) \rightarrow \text{workCell}(s(t), x) = \text{workCell}(t, x)). \end{aligned}$$

Here the formulae $\{\alpha_m\}_{m \in \{L, S, R\}}$ and $\{\beta_m\}_{m \in \{L, S, R\}}$ relate the position of the input and work tape heads at time t with their positions at time $t + 1$:

$$\begin{aligned} \alpha_m &\equiv \begin{cases} \exists x (s(x) = \text{inpHead}@ (t) \wedge \text{inpHead}@ (s(t)) = x) & \text{if } m = L, \\ \text{inpHead}@ (s(t)) = \text{inpHead}@ (t) & \text{if } m = S, \\ \text{inpHead}@ (s(t)) = s(\text{inpHead}@ (t)) & \text{if } m = R, \end{cases} \\ \beta_m &\equiv \begin{cases} \exists x (s(x) = \text{workHead}@ (t) \wedge \text{workHead}@ (s(t)) = x) & \text{if } m = L, \\ \text{workHead}@ (s(t)) = \text{workHead}@ (t) & \text{if } m = S, \\ \text{workHead}@ (s(t)) = s(\text{workHead}@ (t)) & \text{if } m = R. \end{cases} \end{aligned}$$

Accepting Condition.

$$\varphi_{\text{accept}} \equiv \exists t (\text{state}@ (t) = \mathbf{q}_{\text{acc}}).$$

Proof of Correctness of Reduction

To prove that the reduction is correct, we need to establish that:

1. it can be effectively computed,
2. it is sound (i.e. if $\varphi_w \in \text{VALID}_{\text{FOL}}$, then $w \in \text{MP}$),
3. it is complete (i.e. if $w \in \text{MP}$, then $\varphi_w \in \text{VALID}_{\text{FOL}}$).

Question 0.6. 3 Argue that the reduction is a computable function.

Soundness. We assume that φ_w is valid. Starting with this assumption, we will show that w has an accepting run in the Turing machine M . In particular, for the structure

$$\mathcal{A}_{\mathbb{N}, M} = (U_{\mathbb{N}, M}, I_{\mathbb{N}, M}),$$

we must have

$$\mathcal{A}_{\mathbb{N}, M} \models \varphi_w.$$

The universe of the structure $\mathcal{A}_{\mathbb{N}, M}$ is the set of natural numbers.

Question 0.7. 4 Complete the description of the interpretation $I_{\mathbb{N}, M}$ by specifying it for every constant and function symbol in the signature Σ .

Question 0.8. 5 Complete the proof of soundness of the reduction by arguing that the interpretation constructed above can be used to construct a run of the Turing machine M on w that is also an accepting run.

Hint: You may want to use induction to argue that each prefix of the run obeys the semantics of the Turing machine M .

Completeness. Here, we start with the assumption that $w \in \text{MP}$ and argue that φ_w is valid, or equivalently, for every structure $\mathcal{A}$, we have

$$\mathcal{A} \models \varphi_w.$$

If $\mathcal{A}$ does not satisfy one or more of

$$\varphi_{\text{successor}}, \varphi_{\text{consistent}}, \varphi_{\text{init}}, \varphi_{\text{transition}},$$

then clearly $\mathcal{A} \models \varphi_w$. So we only need to argue about those structures $\mathcal{A}$ for which each of

$$\mathcal{A} \models \varphi_{\text{successor}}, \quad \mathcal{A} \models \varphi_{\text{consistent}}, \quad \mathcal{A} \models \varphi_{\text{init}}, \quad \mathcal{A} \models \varphi_{\text{transition}}$$

hold.

Let the sequence of configurations of M on w be

$$c_0, c_1, \dots, c_k$$

where c_k is the accepting configuration.

Question 0.9. 6 Prove completeness of the reduction by formally establishing the above argument. That is, formally show that if $w \in \text{MP}$, then φ_w is a valid FOL formula.

Question 0.10. 7 Prove Theorem 1.

A Resolution Proof System for FOL

As with propositional logic, a resolution-style proof system has also been defined for proving unsatisfiability of FOL formulae. We will recall this proof system in modern terminology and also prove its soundness and completeness using the results we have already learnt in the course so far. As with propositional logic, the resolution proof system for FOL is also catered towards establishing unsatisfiability of a set of formulae Γ (instead of establishing validity as in Hilbert style proof systems) over some signature Σ .

Simplifying assumptions. We will assume that Γ is a countable set of formulae, each of which is presented in prenex normal form. Also, we will assume that all formulae are universally quantified sentences. Further, we will assume that the formulae do not contain the “=” symbol. Finally, we assume that the matrix of every formula $\varphi \in \Gamma$ is a clause, i.e.

$$\varphi = \forall x_1 \forall x_2 \dots \forall x_k \left(\bigvee_{i=1}^n \alpha_i \right),$$

where each α_i is an atomic formula, with

$$\text{FVar}(\alpha_i) \subseteq \{x_1, \dots, x_k\}.$$

Definition 0.11 (Resolution Proof for FOL). A sequence

$$\pi = \varphi_1, \varphi_2, \dots, \varphi_n$$

of sentences over a signature Σ is said to be a resolution proof for a set of universally quantified sentences Γ (over the same signature Σ) if $\varphi_n = \emptyset$ is the empty clause, and for every $1 \leq i \leq n$, one of the following holds:

1. **(Axiom)** $\varphi_i \in \Gamma$.
2. **(Instantiation)** There is a $j < i$ such that $\varphi_i \in \text{Ground}(\varphi_j)$.
3. **(Resolvent)** There are $j, k < i$ such that φ_j and φ_k are ground clauses, and there is a ground atom α such that $\alpha \in \varphi_j$ and $\neg\alpha \in \varphi_k$ and

$$\varphi_i = (\varphi_j \setminus \{\alpha\}) \cup (\varphi_k \setminus \{\neg\alpha\}).$$

Theorem 0.12. The resolution proof system defined above is sound. That is, let Γ be a set of universally quantified sentences, with the assumptions outlined above, over signature Σ . If Γ has a resolution proof, then Γ is unsatisfiable.

Question 0.13. 8 Prove Theorem 2.

Hint: Instead of proving soundness from scratch, use Herbrand's Theorem, together with the soundness of propositional resolution proof system.

Theorem 0.14. The resolution proof system defined above is complete. That is, let Γ be a set of universally quantified sentences, with the assumptions outlined above, over signature Σ . If Γ is unsatisfiable, then there is a resolution proof for Γ .

Question 0.15. 9 Prove Theorem 3.

Hint: Instead of proving completeness from scratch, use Herbrand's Theorem, together with the completeness of propositional resolution proof system.

Question 0.16. 10 Use the soundness and completeness of the FOL resolution proof system to show that

$$\text{VALID}_{\text{FOL}} \in \text{RE}$$

and thus $\text{VALID}_{\text{FOL}}$ is RE-complete.

Small Model Property and FOL Fragments

In the last question, we proved that the validity problem for FOL is RE-complete, thus undecidable. Its dual problem, the satisfiability problem for FOL,

$$\text{SAT}_{\text{FOL}},$$

is coRE-complete, which is also undecidable. In this part, we focus on FOL fragments for which the classical decision problems SAT_{FOL} and $\text{VALID}_{\text{FOL}}$ are decidable.

Definition 0.17 (Small Model Property). A class $\mathcal{C}$ of FOL has the small model property if there is a computable function f such that for every $\phi \in \mathcal{C}$, if ϕ is satisfiable then it has a model of size at most $f(|\phi|)$.

The first FOL fragment is the class of quantifier-free FOL formulae.

Question 0.18. 11 Formally define the quantifier-free fragment of FOL and show that it has the small model property.

Question 0.19. 12 Prove that the satisfiability and validity problems for the quantifier-free fragment of FOL are decidable.

The second FOL fragment is EPR (or Bernays–Schönfinkel) class of FOL formulae, containing function-free formulae with quantifiers of the form

$$\exists^* \forall^*.$$

Question 0.20. 13 Prove that the satisfiability problem for the EPR fragment of FOL is decidable.